Artificial Neural Network from Scratch Using Numpy

1. Introduction : The main aim of the project is to understand how an artificial neural network works. Instead of using a ready-made deep learning library , I used Numpy to implement the basic operations of a neural network.

   For this project I used the iris dataset the model takes four measurements of a flower as input and predicts whether it belongs to setosa , versicolor or virginica.

2. Dataset and Preprocessing : The iris dataset contains three types of flowers and four features: sepal length, sepal width, petal length and petal width.

   I divided the dataset into training , validation and testing sets using stratified splitting . around 70% of the data was used for training 15% for validation , and 15% for testing

   Before training , I standardised the input features using the mean and standard deviation of the training data . I also converted the class labels into one-hot encoded values so that they could be used by the output layer

3. Neural Network Architecture : The Network has four input features , two hidden layers containing 8 and 6 neurons, and an output layer containing three neurons.

   I used sigmoid activation in the hidden layers and softmax in the output layer. The Model was trained using mini-batches of 16 samples and a learning rate off 0.01

   I selected the architecture to keep the network small and while allowing it to learn Relationship between the flower measurements

4. Implementation : I implemented the main neural network operations using numpy . forward propagation calculates the output of each layer . the sigmoid function is used in the hidden layers, while softmax produces probabilities for the three flower classes.

   Cross-entropy loss is used to measure the difference between the predicted probabilities and the actual labels. During backpropagation, the error is propagated backwards through the network to calculate gradients for the weights and biases Gradient descent then updates these parameters to reduce the loss.

   I also recorded the training loss and validation accuracy during training. Early Stopping was included to avoid continuing training when the validation loss stopped Improving.

5. Results : The saved model was tested on 23 samples from the test set. It correctly classified all 23 samples, giving a test accuracy of 100%.

   The confusion matrix showed that all 7 Setosa samples, all 8 Versicolor samples, And all 8 Virginica samples were classified correctly.

   During training, the validation accuracy reached 95.45%. At epoch 1000, the recorded validation accuracy was 90.91%.

   The model performed well on this test split. However, the test set is small, so the same accuracy cannot be guaranteed on other data.

Figure 1: Confusion Matrix of the Neural Network

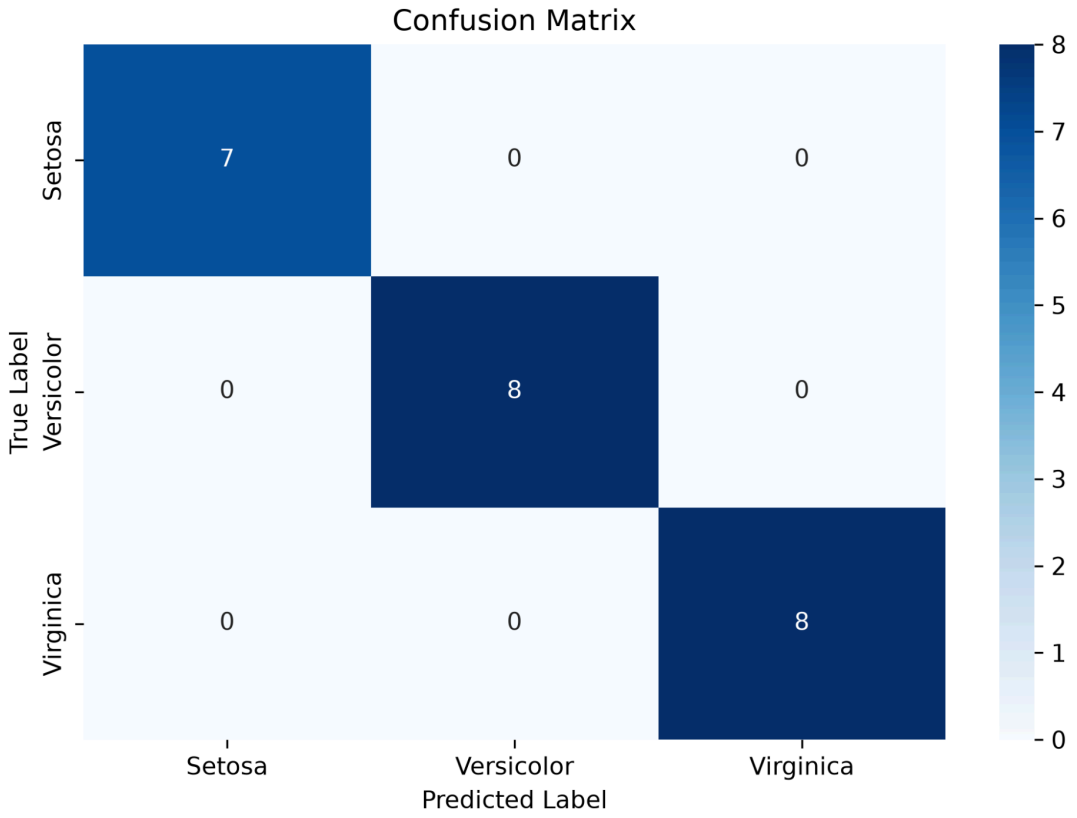
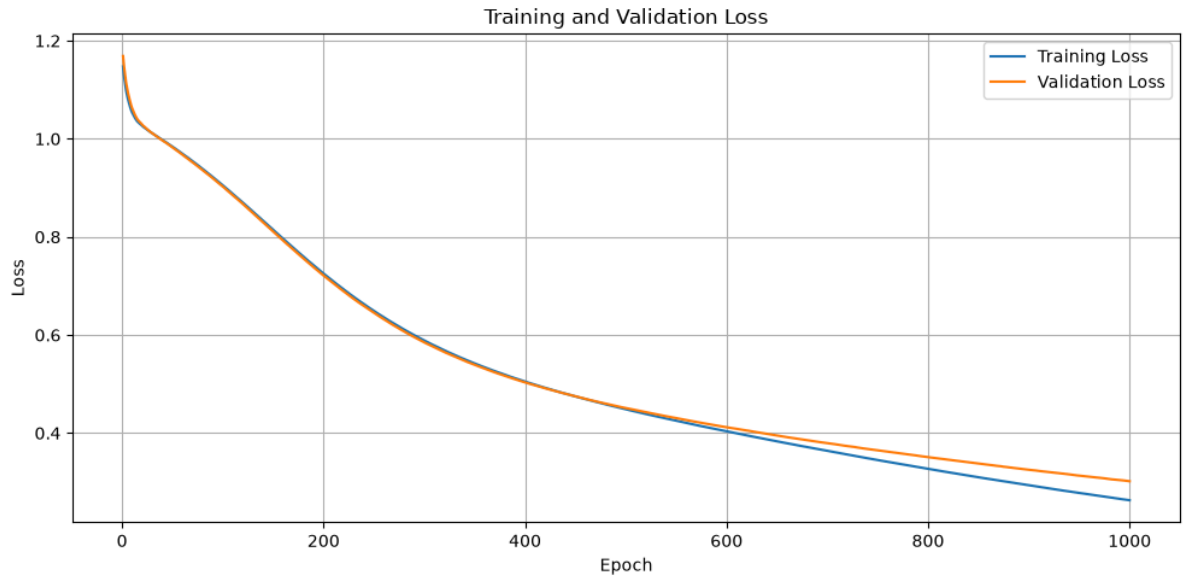


Figure 2: Training Loss During Model Training



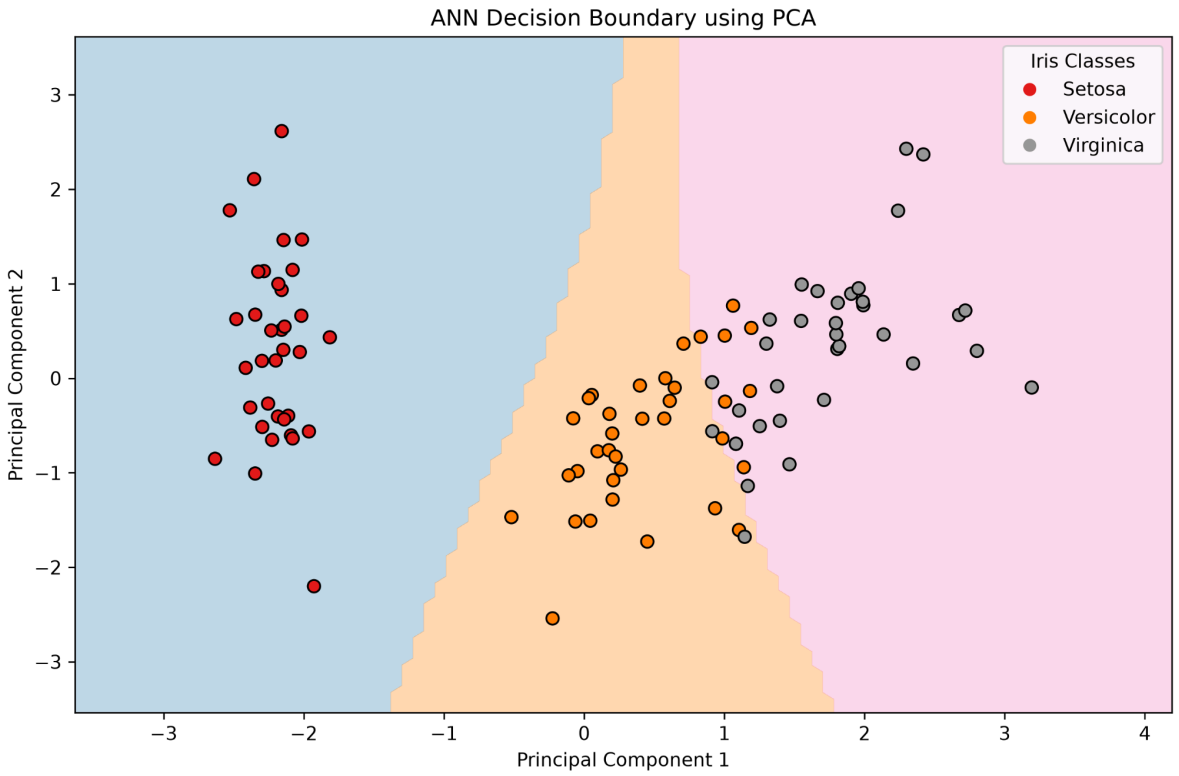
6. Experiment Comparison : I tested different learning rates: 0.001, 0.01, and 0.1. I also compared sigmoid, ReLU, and tanh activation functions and two hidden-layer architectures: 8-6 and 16-8.

   The results showed that the choice of learning rate, activation function, and network size affected the training and validation results. Some configurations achieved the same validation accuracy but had different validation losses.

Learning rate	Activation	Architecture	Validation loss
0.001	tanh	16-8	0.403
0.01	ReLU	16-8	0.162
0.1	ReLU	8-6	0.560
0.1	ReLU	16-8	0.351

   The highest validation accuracy in these experiments was 90.91%. I used both validation accuracy and validation loss to compare the configurations. The detailed results are available in the experiment_results.csv file.

Figure 3: PCA-Based Decision Boundary for iris Classification



7.Conclusion : Through this project, I learned how the main parts of an artificial neural network work together. Implementing forward propagation, backpropagation, and gradient descent helped me understand how the model learns from data.

I also learned how preprocessing, activation functions, and learning rates can affect model performance. The experiments gave me a better understanding of training and evaluating a neural network.